

Optimization of a 2D Convolution

Advanced Computer Architecture Project

Nicolò Galli

Department of Electrical, Computer and Biomedical Engineering
University of Pavia

 https://github.com/nico-log/2D_Conv_Parallel.git

September 15, 2026

Objective:

- Building a CUDA kernel that applies a **3x3 convolutional** filter to an image
- **Exploring** different kernel **optimizations**
- **Measuring** the impact on performance of each **optimization**

Why build a kernel from scratch if optimized solutions already exist?

- **Learn** the basis of CUDA programming and GPU architecture
- **Customized solution**
- **No dependencies** from external huge libraries (embedded system applications)

Convolution for image filtering

Generally, color images are represented as 2D signals with 3 channels (RGB). Given:

- $\mathbf{x}$: input image ($height \times width \times 3$)
- $\mathbf{w}$: filter matrix ($k \times k$)

The convolution $\mathbf{z} = \mathbf{x} * \mathbf{w}$ is computed as follows¹:

$$z_{u,v,c} = \sum_{t=0}^{k-1} \sum_{s=0}^{k-1} w_{s,t} \cdot x_{u+s,v+t,c} \quad (1)$$

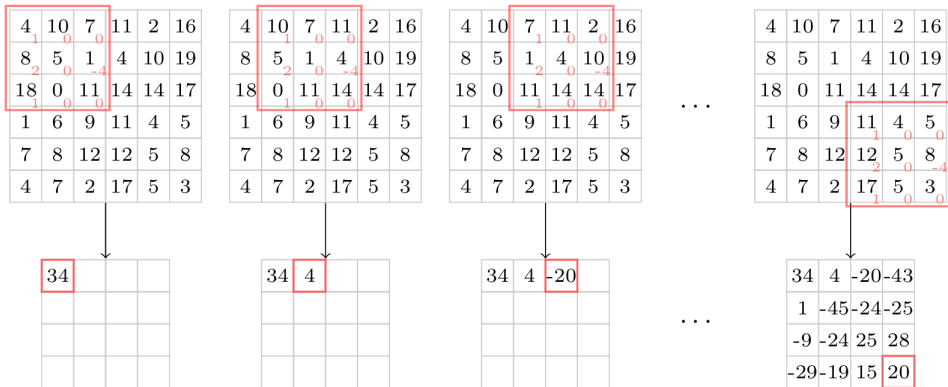
The result is a $(height - k + 1) \times (width - k + 1) \times 3$ image.

The same $k \times k$ spatial filter is applied **independently** to each color channel.

¹Cross-correlation formula commonly used in ML frameworks, not the mathematical definition

Convolution for image filtering

Example of a 2D single-channel convolution 6×6 input image, a 3×3 filter resulting in a 4×4 output image



Padding

Application of same size convolution with zero padding to keep the output size the same as the input size

0	0	0	0	0	0	0
0	x_{00}	x_{01}	x_{02}	x_{03}	x_{04}	0
0	x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	0
0	x_{20}	x_{21}	x_{22}	x_{23}	x_{24}	0
0	x_{30}	x_{31}	x_{32}	x_{33}	x_{34}	0
0	x_{40}	x_{41}	x_{42}	x_{43}	x_{44}	0
0	0	0	0	0	0	0

Figure: Padded input image

z_{00}	z_{01}	z_{02}	z_{03}	z_{04}
z_{10}	z_{11}	z_{12}	z_{13}	z_{14}
z_{20}	z_{21}	z_{22}	z_{23}	z_{24}
z_{30}	z_{31}	z_{32}	z_{33}	z_{34}
z_{40}	z_{41}	z_{42}	z_{43}	z_{44}

Figure: Same size convolution

Zero-padding creates an artificial high-contrast step at the border.

Filters



Test Image ²

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



Blur

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Edge

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

²Sebastian Pena Lambarri - Whaleshark, Maldives

Program usage

A **command line tool** that requires four parameters:

```
./convolution <filter> <iterations> <input_image> <output_image>
```

1. Filter type (edge, sharpen, blur)
2. Number of iterations for benchmarking (0, 100]
3. Input image path
4. Output image path

The program prints different **metrics** about the execution and, after a **validation**, saves the output image(s).

Program assumptions

- **Filter size:** 3x3 (fixed)
- **Image channels:** 3 (RGB) $\rightarrow$ 4 (RGBA)
- **Output image:** jpeg
- **External libraries:** stb_image and stb_image_write header-only libraries

External Libraries

The source code of **header-only libraries** can be directly included in the project by defining macros, **avoiding external dependencies** (suitable for the project).

Program structure

1. **Setup and initialization**
2. **Serial CPU Implementation** (Baseline)
3. **Parallel GPU implementation**
 - **v1:** Naive
 - **v2:** Shared Memory
 - **v3:** + Constant Memory
 - **v4:** + Vectorization
 - **v5:** + loop unrolling (Final kernel)
4. **Validation**
5. **Export and Free Memory**

Initialization (arguments and image)

- **Parsing**: command-line argument validation with error handling
- **Loading image**: pixel values decoded to RAM as a flattened 1D array
- **RGBA layout**: virtual alpha channel added for vectorized memory accesses:
[R, G, B, A, R, G, B, A, ...]
- **Output buffers**: dynamic allocation of host buffers for CPU-GPU validation

Validation

The validation function **compares** each pixel between CPU and GPU results within a tolerance of 1 to absorb rounding/casting differences.

Initialization (Grid and Blocks)

2D Block

```
dim3 threadsPerBlock(16, 16);
```

- $16 \times 16 = 256$ threads
- Bidimensional block: match between pixel and thread organization

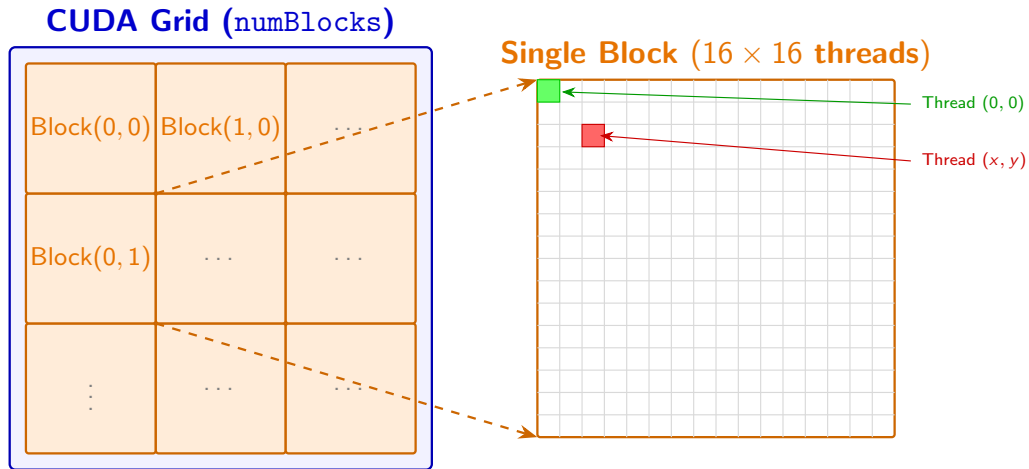
2D Grid

```
dim3 numBlocks(  
    (width + threadsPerBlock.x - 1) / threadsPerBlock.x,  
    (height + threadsPerBlock.y - 1) / threadsPerBlock.y);
```

- Covers entire image even if dimensions are not multiples of 16 (**ceiling**)

$$\lceil \frac{width}{16} \rceil \times \lceil \frac{height}{16} \rceil$$

Initialization (Grid and Blocks)



Benchmarking protocol

Implementation	Warm-up	Measured Runs (ms)	Tool
CPU Baseline	1 run	1 run	<code>std::chrono</code>
GPU Kernels	1 run	100 runs	<code>cudaEventRecord()</code>

To get stable and less noisy measurements:

- **Warm-up:** first launch not measured to discard initialization overhead
- **Multiple-runs:** kernels are vulnerable to possible latencies due to their low execution times

GPU times measure kernel execution only and exclude host-device transfers and memory allocation.

Benchmarking

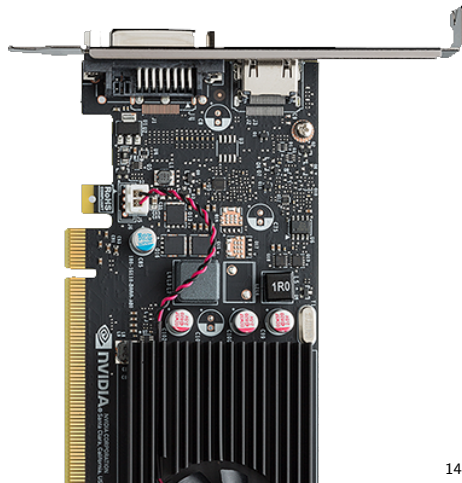
Derived metrics:

- Average Execution time (ms)
- Speed-Up (CPU/GPU)
- Throughput (MP/s)

Test Environment:

- **GPU:** NVIDIA GeForce GT 1030
- **Architecture:** Pascal
- **Memory:** 2GB GDDR5, 64-bit bus
- **Peak Compute:** 1.13 TFLOP/s
- **Peak Bandwidth:** 48 GB/s
- **CUDA:** version 12.6

Input image: 4344×2896 , 4 channels (RGBA)



0. Serial CPU implementation

- Single thread execution
- 3 nested for-loops (height, width, channel)
- Implicit zero padding via boundary checks
- Result clamped to $[0, 255]$
- Alpha channel set to 255 (excluded from loop computations)

Performance (edge filter):

Kernel	Time (ms)	Throughput (MP/s)	Speedup
CPU (Baseline)	297.55	42.28	1.00x

1. Parallel GPU implementation (naive)

Memory management:

- **Global memory** exclusive usage.
- **cudaMalloc**: allocate device for buffers input image, output image and filter matrix
- **cudaMemcpy**:
 - **Host** → **Device**: input image and filter (before kernel launch)
 - **Device** → **Host**: output image (after kernel launch)
- **cudaFree**: deallocate GPU buffers

Coordinates

```
int x = blockIdx.x * blockDim.x + threadIdx.x;  
int y = blockIdx.y * blockDim.y + threadIdx.y;
```

Each thread computes one output pixel identified using coordinates:

→ Spatial loops are no longer necessary (only channel loop)

1. Parallel GPU implementation (naive)

Problem

To compute the convolution, each thread reads 9×3 values from the global memory. The same pixel is loaded redundantly by multiple adjacent threads.

Global Memory

Primary, largest storage space on a GPU. Accessible by all threads and the CPU, it holds massive datasets but is **slower to access**.

2. Parallel GPU implementation (shared memory)

Shared Memory

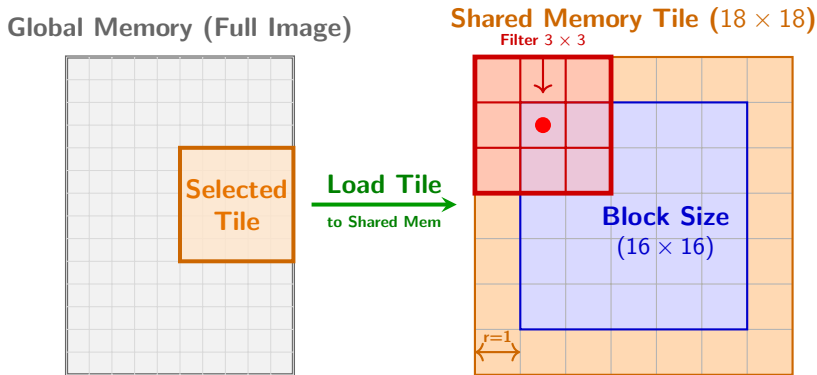
On-chip programmable memory, private for each block and **shared for each thread in a block**. Higher bandwidth and lower latency than the global memory.

Kernel **phases**:

- The input image is copied from global to **shared memory**
- `__syncthreads()`
- The pixels are read from the shared memory and the convolution operation is computed

Halo pixels: threads near the border need pixels outside the block to compute the convolution → Shared memory tile: $18 \times 18 = 324$ pixels

2. Parallel GPU implementation (shared memory)



Problem

The filter matrix is still accessed by a **global memory** pointer

3. Parallel GPU implementation (shared memory + constant memory)

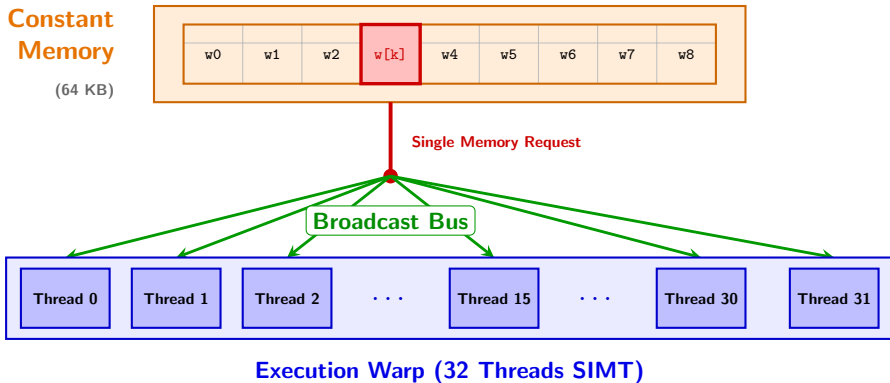
Constant Memory

Small, **read-only** region of GPU device memory optimized for **broadcasting** identical data to multiple threads. Used to store fixed data that doesn't change during execution.

Memory management:

- 3×3 filter matrix (36 bytes) copied via `cudaMemcpyToSymbol()`
- All threads in a warp access the same filter coefficient at each iteration
- Constant memory can efficiently **broadcast** the same coefficient to all threads in a warp

3. Parallel GPU implementation (shared memory + constant memory)



Problem

Loading 3 channels separately requires (conceptually) 3 load instructions per pixel with a 4-byte stride between threads → high instruction count and scheduler pressure.

4. Parallel GPU implementation (+ vectorization)

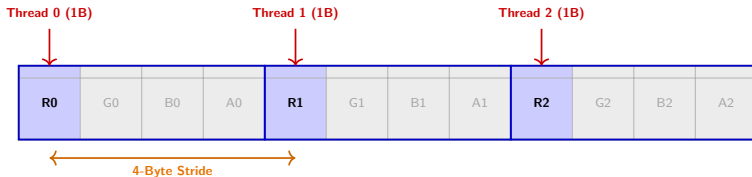
- Each thread loads one complete RGBA pixel using a 4-byte uchar4 access.
- Reduction of explicit memory operations
- Memory coalescing efficiency improved
- **Drawback:** Unused alpha byte

Memory Coalescing

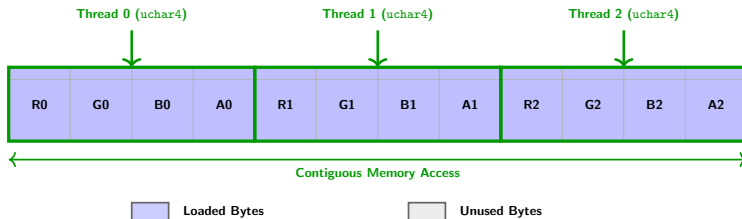
When a 32-thread warp accesses global memory, the hardware tries to **coalesce** the individual memory requests into the smallest possible number of **memory transactions**. Coalescing efficiency depends on the address pattern and alignment of the accesses.

4. Parallel GPU implementation (+ vectorization)

Scalar Memory Access



Vectorized Memory Access



Problem

Filter **loop overhead** (increment variable, check condition, branch)

5. Parallel GPU implementation (+ unrolling)

`#pragma unroll`

Compiler **directive** instructing `nvcc` to **unroll** the loop body at assembly level when iteration bounds are known at compile time.

Advantages of loop unrolling:

- Reduces loop-control overhead
- May enable better instruction scheduling

Compiler Impact

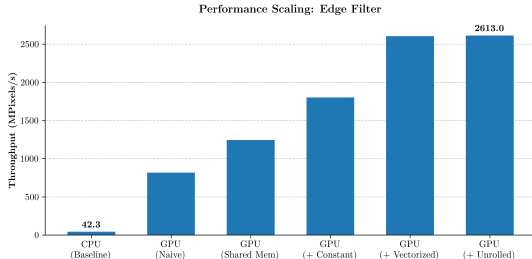
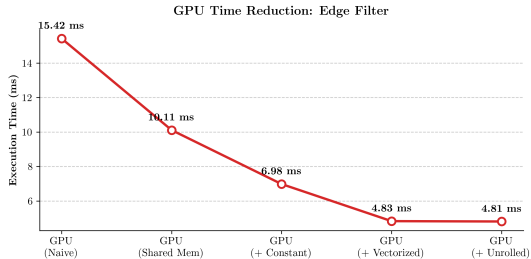
Loop bounds are compile-time constants and the loop is very small (3×3): `nvcc` may already perform substantial optimization → **low expected impact**

Table: Performance Summary - Edge Filter

Kernel	Time (ms)	Throughput (MP/s)	Speedup
CPU (Baseline)	297.55	42.28	1.00x
GPU (Naive)	15.42	815.98	19.30x
GPU (Shared Mem)	10.11	1244.06	29.42x
GPU (+ Constant)	6.98	1802.84	42.64x
GPU (+ Vectorized)	4.83	2605.57	61.63x
GPU (+ Unrolled)	4.81	2613.04	61.80x

Blur and Sharpen show nearly identical performance because they share the same computational structure and memory access pattern.

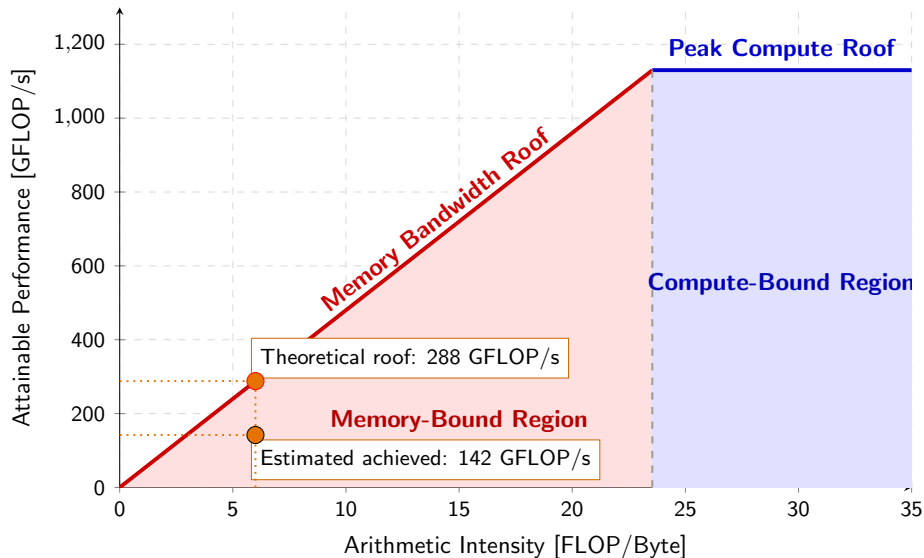
Performance Scaling Analysis



Most of the performance gains come from **memory-oriented** optimizations.

- Shared and constant memory reduce costly global-memory accesses
- Vectorization improves access efficiency and memory coalescing.

Roofline Model



Analytical Assumptions

- Uses nominal datasheet specifications rather than sustained runtime values.
- Each block transfers a full tile from DRAM
(18×18 read, 16×16 write = 2320 Byte).
- Assumes direct global memory reads, neglecting potential L1/L2 cache hits..

Performance Gap Analysis (142 **GFLOP/s** vs. 288 **GFLOP/s**)

- 4 bytes are loaded per pixel via uchar4, but only 3 channels (RGB) generate FLOPs (Unused Alpha Channel)
- Significant pipeline cycles are consumed by non-FLOP instructions (indexing, conversions, synchronization, clamping, ...).
- Effective memory bandwidth (estimated: 23.7 GB/s) does not saturate the nominal peak bandwidth (48 GB/s).

Conclusions

- $\approx 61\times$ speed-up achieved over serial CPU baseline
- According to the analytical roofline model, the kernel is **memory-bound**:

$$AI_{kernel} < AI_{ridge} \rightarrow 6 < 23.5 \text{ FLOP/B}$$

- Explains why memory optimizations drove the speed-up
- Unrolling gains were minimal due to compiler optimizations on static loops

Essential Future Developments

- Hardware profiling via **NVIDIA Nsight Compute**
- Benchmark with modern hardware
- comparison with already existing libraries
- Studying memory behaviour with larger filters (5×5 , 7×7)
- Experimental exploration of block dimensions (8×8 , 32×32)

Bibliography

- Notes on Parallel Programming (2025) by Francesco Leporati
- Notes on Advanced Computer Architecture (2026) by Emanuele Torti
- Notes on Machine Learning (2025) by Claudio Cusano
- CUDA Programming Guide, Release 13.3, NVIDIA Corporation (May 27, 2026)
- NVIDIA GeForce GT 1030 documentation
(<https://www.nvidia.com/en-us/geforce/graphics-cards/gt-1030/>)
- TechPowerUp GPU Database, 'NVIDIA GeForce GT 1030 Specs', 2017
(<https://www.techpowerup.com/gpu-specs/geforce-gt-1030.c2954>)
- Mouna Afif, Yahia Said, Mohamed Atri, " Efficient 2D Convolution Filters Implementations on Graphics Processing Unit Using NVIDIA CUDA ", International Journal of Image, Graphics and Signal Processing(IJIGSP), Vol.10, No.8, pp. 1-8, 2018.DOI: 10.5815/ijigsp.2018.08.01
- stb repository (<https://github.com/nothings/stb>)
- Test Image: Sebastian Pena Lambarri - Whaleshark, Maldives